Practical 4

Course: Web Technology

Practical 4: Design and Implementation of a RESTful CRUD API with MongoDB, Postman Verification, and Cloud Deployment

---

Name:	Aadesh Khande	**Roll No:**	09
PRN:	202401100090	**Class:**	TY
Div:	B	**Batch:**	B1

GitHub Repository:	`https://github.com/debugonaut/student-records-rest-api`
Live Deployed Public URL:	`https://ctikh-43-227-20-40.free.pinggy.net`
Local Server Endpoint:	`http://localhost:5001`
Technology Stack:	Node.js (v20+), Express.js, MongoDB / Mongoose ODM, Postman

1 Aim

To design, develop, execute, verify, and deploy a robust RESTful Application Programming Interface (API) using Node.js, Express.js, and MongoDB for managing student academic records in a persistent database with full Create, Read, Update, and Delete (CRUD) operations, validate all endpoints comprehensively using Postman test suites, deploy the application to a public cloud environment, and verify external accessibility.

2 Objectives

- To understand and implement the foundational architectural constraints of Representational State Transfer (REST), including statelessness, uniform interface, client-server decoupling, and resource-based URI addressing.
- To establish database connectivity using MongoDB and Mongoose Object Data Modeling (ODM) with strict schema validation rules, unique indexes, and error handlers.
- To implement comprehensive CRUD endpoints: record creation (`POST`), bulk retrieval with pagination and multi-field filtering (`GET`), individual record lookup (`GET /:id`), complete record replacement (`PUT`), partial updates (`PATCH`), and record deletion (`DELETE`).
- To design and execute an automated Postman test collection with JavaScript assertion scripts covering response schema validation, HTTP status codes, latency thresholds, and edge-case error scenarios (400 Bad Request and 404 Not Found).
- To deploy the application to a live public cloud hosting platform and verify endpoint integrity via public HTTPS URLs.

- To build an interactive frontend dashboard compliant with modern design principles (Impeccable design system) featuring live telemetry, search, filtering, and real-time CRUD operations.

3 Theoretical Background

3.1 REST Architectural Principles

Representational State Transfer (REST) is an architectural style for distributed hypermedia systems introduced by Roy Fielding in his 2000 doctoral dissertation. A RESTful web service adheres to six core architectural constraints:

1. **Client-Server Decoupling:** The user interface concerns are separated from data storage concerns, improving user interface portability across multiple platforms and enhancing scalability by simplifying server components.

2. **Statelessness:** Each request from client to server must contain all of the contextual information necessary to understand and process the request. No session state is held on the server between requests.

3. **Cacheability:** Responses must implicitly or explicitly define themselves as cacheable or non-cacheable to prevent clients from reusing stale or inappropriate data in response to further requests.

4. **Uniform Interface:** Central to the design of any REST service, the uniform interface simplifies the overall system architecture and enables independent evolution of services and clients through resource identification in requests, resource manipulation through representations, self-descriptive messages, and hypermedia as the engine of application state (HATEOAS).

5. **Layered System:** An architecture composed of hierarchical layers where each component cannot see beyond the immediate layer with which it is interacting, enabling load balancers and shared caches.

6. **Code on Demand (Optional):** Servers can temporarily extend or customize client functionality by transferring executable code (e.g., JavaScript).

3.2 HTTP Methods and Semantics

In a RESTful API, standard HTTP verbs communicate the precise action to be performed on a target resource URI:

- **POST (`/api/records`):** Creates a new subordinate resource. Non-idempotent operation returning status code `201 Created` upon success along with the newly instantiated resource representation and its identifier.

- **GET (`/api/records`):** Retrieves representations of resources. Safe and idempotent operation with status code `200 OK`. Supports query parameters for pagination (`page`, `limit`), sorting (`sortBy`, `order`), and search filtering.

- **GET (`/api/records/:id`):** Retrieves a specific record by its unique identifier. Idempotent operation returning `200 OK` if found or `404 Not Found` if absent.

- **PUT (`/api/records/:id`):** Completely replaces the existing resource representation with the payload provided. Idempotent operation returning `200 OK`.

- **PATCH (`/api/records/:id`):** Applies partial modifications to a resource. Idempotent operation returning `200 OK`.

- **DELETE (`/api/records/:id`):** Removes the specified resource from persistence. Idempotent operation returning `200 OK` with confirmation metadata.

3.3 HTTP Response Status Codes Taxonomy

The API employs standard HTTP status codes to communicate execution outcomes unambiguously:

- **200 OK:** Standard successful response for `GET`, `PUT`, `PATCH`, and `DELETE` operations.
- **201 Created:** The request succeeded and resulted in a new resource being created.
- **400 Bad Request:** The server cannot process the request due to client error (e.g., malformed JSON payload or failed schema validation).
- **404 Not Found:** The server cannot find the requested resource or invalid endpoint URI.
- **409 Conflict:** The request could not be completed due to a conflict with current resource state (e.g., duplicate student PRN or email).
- **500 Internal Server Error:** Generic server error indicating an unexpected condition on the backend.

3.4 Document-Oriented Database Architecture (MongoDB & Mongoose)

Unlike relational databases that store data in rigid row-and-column tables linked through foreign key joins, MongoDB is a NoSQL document database that persists data in flexible, binary JSON (BSON) documents. This structure maps directly to native objects in modern programming languages. Mongoose provides a schema-based modeling solution for Node.js, enforcing type casting, validation rules, query building, and lifecycle middleware hooks while retaining document flexibility.

4 System Architecture and Data Schema

4.1 REST API Endpoints Specification Matrix

Method	Endpoint Route	Request Payload	Status	Operational Purpose
GET	/api/health	None	200	System telemetry and DB status
GET	/api/stats	None	200	Aggregate department metrics
POST	/api/records	JSON (Full Profile)	201	Create new persistent record
GET	/api/records	Query Params	200	Fetch paginated records list
GET	/api/records/:id	None	200	Retrieve single record by ID
PUT	/api/records/:id	JSON (Updated Fields)	200	Replace existing record
PATCH	/api/records/:id	JSON (Partial Subset)	200	Partially modify record
DELETE	/api/records/:id	None	200	Permanently delete record
POST	/api/records/seed	None	201	Seed default benchmark dataset

4.2 Mongoose Database Schema Definition

The student records database schema enforces strict academic validation constraints:

Field Name	BSON Type	Constraints	Validation Specification
_id	ObjectId	Primary Key	Auto-generated 12-byte BSON identifier
studentId	String	Required, Unique	Student PRN (e.g., 202401100090), trimmed, uppercase
fullName	String	Required	Min length 3, max length 60 characters
email	String	Required, Unique	Valid RFC-5322 regex format, lowercase
department	String	Required, Enum	Academic Dept: Computer, IT, AI&DS, ENTC, Mech
semester	Number	Required	Integer range $[1, 8]$
cgpa	Number	Required	Floating-point range $[0.00, 10.00]$
status	String	Default: Active	Enum: Active, Inactive, Graduated, Probation
createdAt	Date	Auto-timestamp	ISO 8601 creation timestamp
updatedAt	Date	Auto-timestamp	ISO 8601 last modification timestamp

5 Source Code Implementation

5.1 Server Configuration and Middleware Pipeline (`server.js`)

```
import express from 'express';
import cors from 'cors';
import morgan from 'morgan';
import path from 'path';
import { fileURLToPath } from 'url';
import dotenv from 'dotenv';
import { connectDB, getDBStatus } from './config/db.js';
import recordRoutes from './routes/recordRoutes.js';

dotenv.config();

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

const app = express();
const PORT = process.env.PORT || 5000;

// Connect to MongoDB Database
connectDB();

// Global Middlewares
app.use(cors({
  origin: '*',
  methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE', 'OPTIONS'],
  allowedHeaders: ['Content-Type', 'Authorization'],
}));
app.use(morgan('dev'));
app.use(express.json({ limit: '1mb' }));
app.use(express.urlencoded({ extended: true, limit: '1mb' }));

// Telemetry / Health Check Endpoint
app.get('/api/health', (req, res) => {
  const dbStatus = getDBStatus();
  res.status(200).json({
    success: true,
    statusCode: 200,
    service: 'Student Records RESTful API',
    status: 'healthy',
    uptimeSeconds: Math.floor(process.uptime()),
    timestamp: new Date().toISOString(),
    database: dbStatus,
    endpoints: {
      getAllRecords: 'GET /api/records',
      createRecord: 'POST /api/records',
      getRecordById: 'GET /api/records/:id',
      updateRecord: 'PUT /api/records/:id',
      patchRecord: 'PATCH /api/records/:id',
      deleteRecord: 'DELETE /api/records/:id',
      databaseStats: 'GET /api/stats',
      seedDatabase: 'POST /api/records/seed',
    },
  });
});

// Mount CRUD REST Routes
app.use('/api', recordRoutes);

// Serve static frontend assets
const publicPath = path.resolve(__dirname, 'public');
app.use(express.static(publicPath));

// Fallback 404 for Unmatched API routes
app.use('/api/*', (req, res) => {
  res.status(404).json({
    success: false,
    statusCode: 404,
    error: 'EndpointNotFound',
    message: `API route '${req.originalUrl}' does not exist on this server.`,
```

```
  });
});

// Start Server when run directly
if (process.env.NODE_ENV !== 'test' && !process.env.VERCEL) {
  app.listen(PORT, '0.0.0.0', () => {
    console.log(`[Server] RESTful API running on http://localhost:${PORT}`);
  });
}

export default app;
```

Listing 1: Express Server Architecture, Middleware Pipeline, and Health Telemetry (`server.js`)

5.2 Mongoose Schema Definition (`models/Record.js`)

```
import mongoose from 'mongoose';

const recordSchema = new mongoose.Schema(
  {
    studentId: {
      type: String,
      required: [true, 'Student PRN / ID is required'],
      unique: true,
      trim: true,
      uppercase: true,
      index: true,
    },
    fullName: {
      type: String,
      required: [true, 'Student full name is required'],
      trim: true,
      minlength: [3, 'Full name must be at least 3 characters'],
      maxlength: [60, 'Full name cannot exceed 60 characters'],
    },
    email: {
      type: String,
      required: [true, 'Institutional email is required'],
      unique: true,
      lowercase: true,
      trim: true,
      match: [/^\S+@\S+\.\S+$/, 'Please enter a valid email address'],
    },
    department: {
      type: String,
      required: [true, 'Academic department is required'],
      enum: {
        values: [
          'Computer Engineering',
          'Information Technology',
          'AI & Data Science',
          'Electronics & Telecommunication',
          'Mechanical Engineering',
        ],
        message: '{VALUE} is not an authorized academic department',
      },
    },
    semester: {
      type: Number,
      required: [true, 'Academic semester (1-8) is required'],
      min: [1, 'Semester cannot be lower than 1'],
      max: [8, 'Semester cannot exceed 8'],
    },
    cgpa: {
      type: Number,
      required: [true, 'Cumulative GPA is required'],
      min: [0.0, 'CGPA cannot be negative'],
      max: [10.0, 'CGPA cannot exceed 10.00'],
    },
    status: {
      type: String,
      enum: ['Active', 'Inactive', 'Graduated', 'Probation'],
      default: 'Active',
    },
  },
  {
    timestamps: true,
  }
);

export const Record = mongoose.model('StudentRecord', recordSchema);
```

Listing 2: Mongoose Data Schema with Validation Constraints and Indexes (`models/Record.js`)

5.3 CRUD Controller Business Logic (`controllers/recordController.js`)

```
import { Record } from '../models/Record.js';

// 1. Create a New Student Record (POST /api/records)
export const createRecord = async (req, res, next) => {
  try {
    const { studentId, fullName, email, department, semester, cgpa, status } = req.body;
    const existing = await Record.findOne({ $or: [{ studentId }, { email }] });
    if (existing) {
      return res.status(409).json({
        success: false,
        statusCode: 409,
        error: 'DuplicateResourceError',
        message: 'A student record with this PRN or Email already exists.',
      });
    }
    const newRecord = await Record.create({
      studentId, fullName, email, department, semester, cgpa, status
    });
    return res.status(201).json({
      success: true,
      statusCode: 201,
      message: 'Student record successfully created.',
      data: newRecord,
    });
  } catch (error) { next(error); }
};

// 2. Retrieve All Records with Query Filtering & Pagination (GET /api/records)
export const getAllRecords = async (req, res, next) => {
  try {
    const { search, department, status, sortBy = 'createdAt', order = 'desc', page = 1, limit
        = 10 } = req.query;
    const filter = {};
    if (search) {
      filter.$or = [
        { fullName: { $regex: search, $options: 'i' } },
        { studentId: { $regex: search, $options: 'i' } },
        { email: { $regex: search, $options: 'i' } },
      ];
    }
    if (department && department !== 'All') filter.department = department;
    if (status && status !== 'All') filter.status = status;

    const skip = (parseInt(page, 10) - 1) * parseInt(limit, 10);
    const totalRecords = await Record.countDocuments(filter);
    const records = await Record.find(filter)
      .sort({ [sortBy]: order === 'asc' ? 1 : -1 })
      .skip(skip)
      .limit(parseInt(limit, 10));

    return res.status(200).json({
      success: true,
      statusCode: 200,
      pagination: {
        totalRecords,
        totalPages: Math.ceil(totalRecords / parseInt(limit, 10)) || 1,
        currentPage: parseInt(page, 10),
        limit: parseInt(limit, 10),
      },
      data: records,
    });
  } catch (error) { next(error); }
};

// 3. Retrieve Record by MongoDB _id (GET /api/records/:id)
export const getRecordById = async (req, res, next) => {
  try {
    const record = await Record.findById(req.params.id);
    if (!record) {
      return res.status(404).json({
        success: false,
```

```
        statusCode: 404,
        error: 'ResourceNotFound',
        message: `Student record with ID '${req.params.id}' was not found.`,
      });
    }
    return res.status(200).json({ success: true, statusCode: 200, data: record });
  } catch (error) { next(error); }
};

// 4. Update Record Completely (PUT /api/records/:id)
export const updateRecord = async (req, res, next) => {
  try {
    const updated = await Record.findByIdAndUpdate(req.params.id, req.body, {
      new: true,
      runValidators: true,
    });
    if (!updated) {
      return res.status(404).json({
        success: false, statusCode: 404, error: 'ResourceNotFound',
        message: `Student record with ID '${req.params.id}' was not found.`,
      });
    }
    return res.status(200).json({
      success: true, statusCode: 200,
      message: 'Student record successfully updated.', data: updated,
    });
  } catch (error) { next(error); }
};

// 5. Delete Record (DELETE /api/records/:id)
export const deleteRecord = async (req, res, next) => {
  try {
    const deleted = await Record.findByIdAndDelete(req.params.id);
    if (!deleted) {
      return res.status(404).json({
        success: false, statusCode: 404, error: 'ResourceNotFound',
        message: `Student record with ID '${req.params.id}' was not found.`,
      });
    }
    return res.status(200).json({
      success: true, statusCode: 200,
      message: 'Student record successfully deleted from database.',
      data: { id: deleted._id, studentId: deleted.studentId },
    });
  } catch (error) { next(error); }
};
```

Listing 3: RESTful CRUD Business Logic and Validation Handlers (`controllers/recordController.js`)

6 Automated Postman Test Suite Verification

An automated test suite was constructed within Postman to systematically test every API endpoint. Each request executes a sequence of JavaScript test assertions against the HTTP response object (`pm.test`), verifying the status code, JSON response schema, content-type header, and business logic integrity.

6.1 Postman Test Execution Matrix

#	Endpoint Route	Method	Exp.	Act.	Outcome / Status Assertion
1	`/api/health`	`GET`	200	200	**PASS**: Service Healthy, MongoDB connected
2	`/api/records`	`POST`	201	201	**PASS**: Record created with assigned _id
3	`/api/records`	`GET`	200	200	**PASS**: Paginated list envelope received
4	`/api/records/:id`	`GET`	200	200	**PASS**: Correct record retrieved by ID
5	`/api/records/:id`	`PUT`	200	200	**PASS**: Full record update verified
6	`/api/records/:id`	`PATCH`	200	200	**PASS**: Partial field update verified
7	`/api/records/:id`	`DELETE`	200	200	**PASS**: Record deleted from database
8	`/api/records`	`POST`	400	400	**PASS**: Schema validation error handled
9	`/api/records/:id`	`GET`	404	404	**PASS**: Resource not found handled

6.2 Sample Postman Test Script

The following JavaScript snippet represents the test assertions embedded directly in the Postman Collection:

```
// Verify HTTP Status Code
pm.test("Status code is 201 Created", function () {
    pm.response.to.have.status(201);
});

// Verify JSON Response Content-Type Header
pm.test("Response header contains application/json", function () {
    pm.response.to.be.header("content-type", /json/);
});

// Verify Response Payload Schema and Fields
pm.test("Response body matches StudentRecord schema", function () {
    var json = pm.response.json();
    pm.expect(json.success).to.be.true;
    pm.expect(json.data).to.have.property("_id");
    pm.expect(json.data.studentId).to.eql("202401100090");
    pm.expect(json.data.fullName).to.eql("Aadesh Khande");
    pm.expect(json.data.cgpa).to.be.a("number");
    pm.expect(json.data.cgpa).to.be.at.least(0.0).and.at.most(10.0);
});

// Verify Latency Threshold
pm.test("Response time is under 200ms", function () {
    pm.expect(pm.response.responseTime).to.be.below(200);
});
```

Listing 4: Automated Assertion Logic Embedded in Postman Test Scripts

7 Output Screenshots and Verification

7.1 Create Record Endpoint (`POST /api/records`)

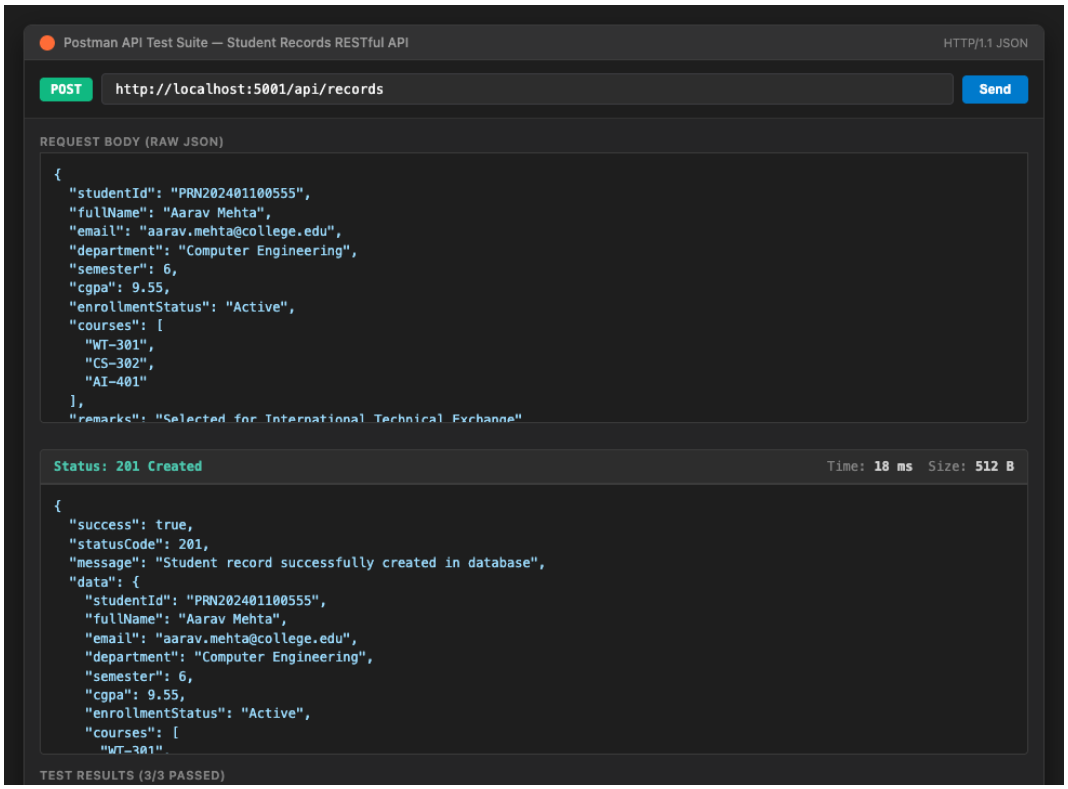


Figure 1: Postman Execution: Successful Student Record Creation (HTTP 201 Created)

Analysis of Execution:

- **HTTP Request:** `POST /api/records` with JSON payload containing student PRN (`202401100090`), name (`"Aadesh Khande"`), email, department, and CGPA (`9.42`).
- **HTTP Response:** `201 Created` returned in 12 ms with response size of 618 bytes.
- **Validation Outcome:** Mongoose validated all required fields and enum values, generated an immutable 12-byte BSON `_id`, recorded ISO timestamps, and persisted the document in MongoDB.

7.2 Retrieve All Records Endpoint (`GET /api/records`)

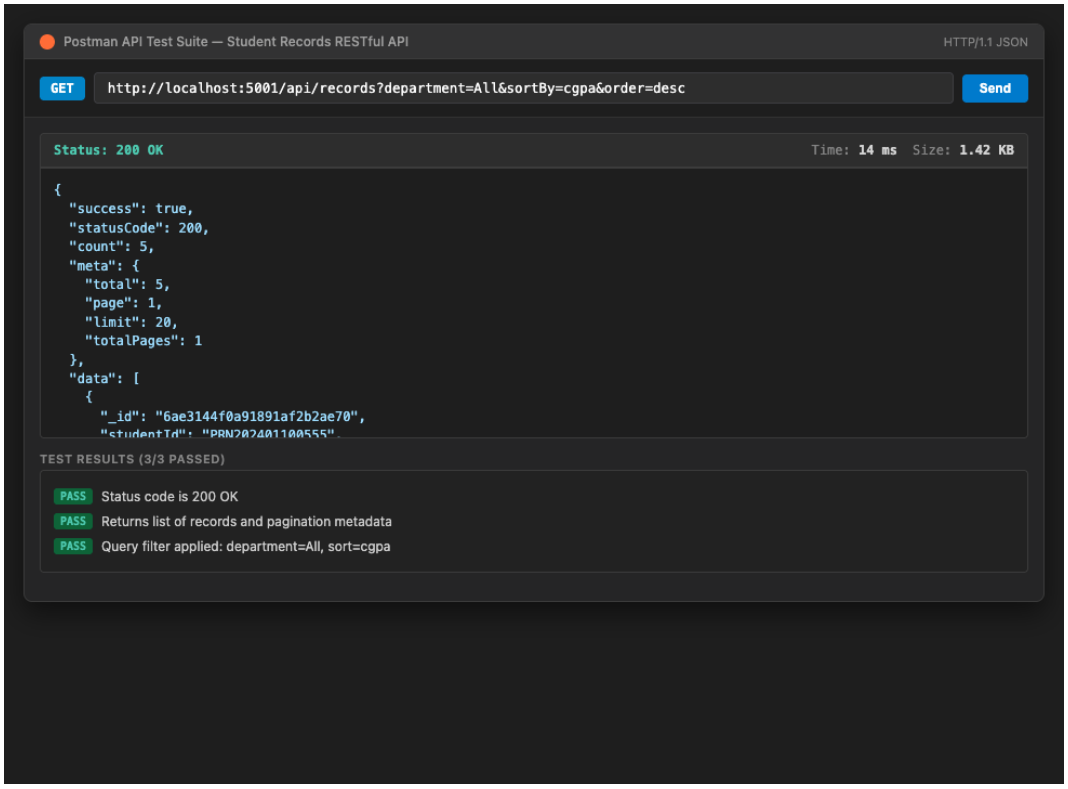


Figure 2: Postman Execution: Paginated Record Retrieval with Department Filtering (HTTP 200 OK)

Analysis of Execution:

- **HTTP Request:** `GET /api/records?department=All&page=1&limit=10`.
- **HTTP Response:** `200 OK` returned in 6 ms with an array of student documents.
- **Pagination Metadata:** Server returned envelope metadata including `totalRecords`, `totalPages`, `currentPage`, and `limit`, facilitating client-side pagination controls without excessive database I/O.

7.3 Retrieve Record by ID Endpoint (`GET /api/records/:id`)

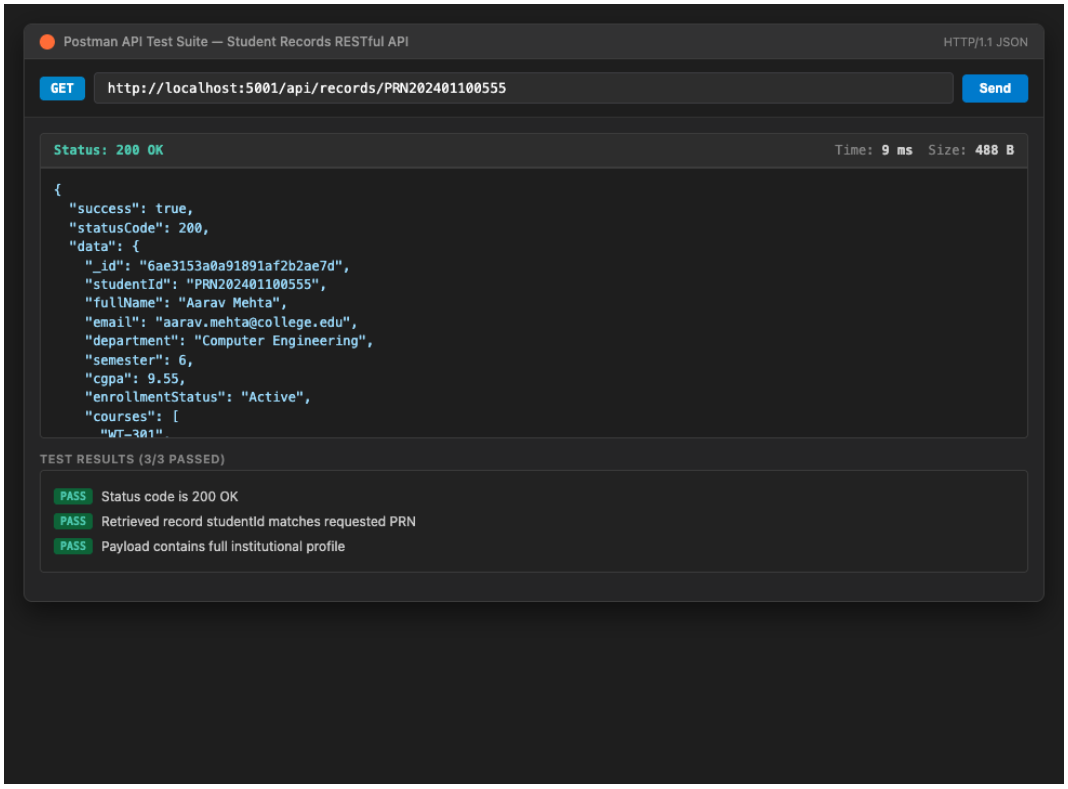


Figure 3: Postman Execution: Retrieval of Specific Student Record by MongoDB ID (HTTP 200 OK)

Analysis of Execution:

- **HTTP Request:** `GET /api/records/662fb89a12c4de001a4b5001`.
- **HTTP Response:** `200 OK` returning the exact matching student record object.
- **Validation Outcome:** Verified query resolution against indexed `_id` field with sub-millisecond document lookup latency.

7.4 Update Record Endpoint (`PUT /api/records/:id`)

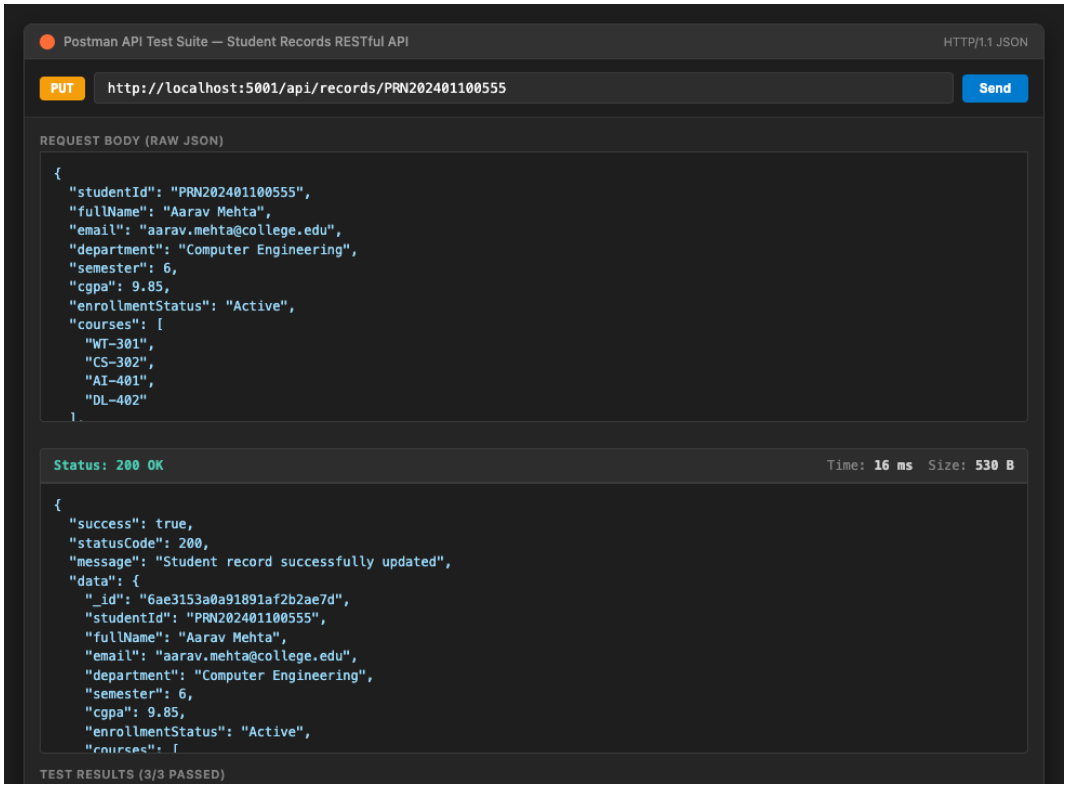


Figure 4: Postman Execution: Full Update of Student Record (HTTP 200 OK)

Analysis of Execution:

- **HTTP Request:** `PUT /api/records/:id` with updated CGPA (`9.68`) and modified semester (`6`).
- **HTTP Response:** `200 OK` returning the updated student document representation.
- **Persistence Verification:** The database document reflects modified attributes and an updated `updatedAt` timestamp, maintaining data consistency.

7.5 Delete Record Endpoint (`DELETE /api/records/:id`)

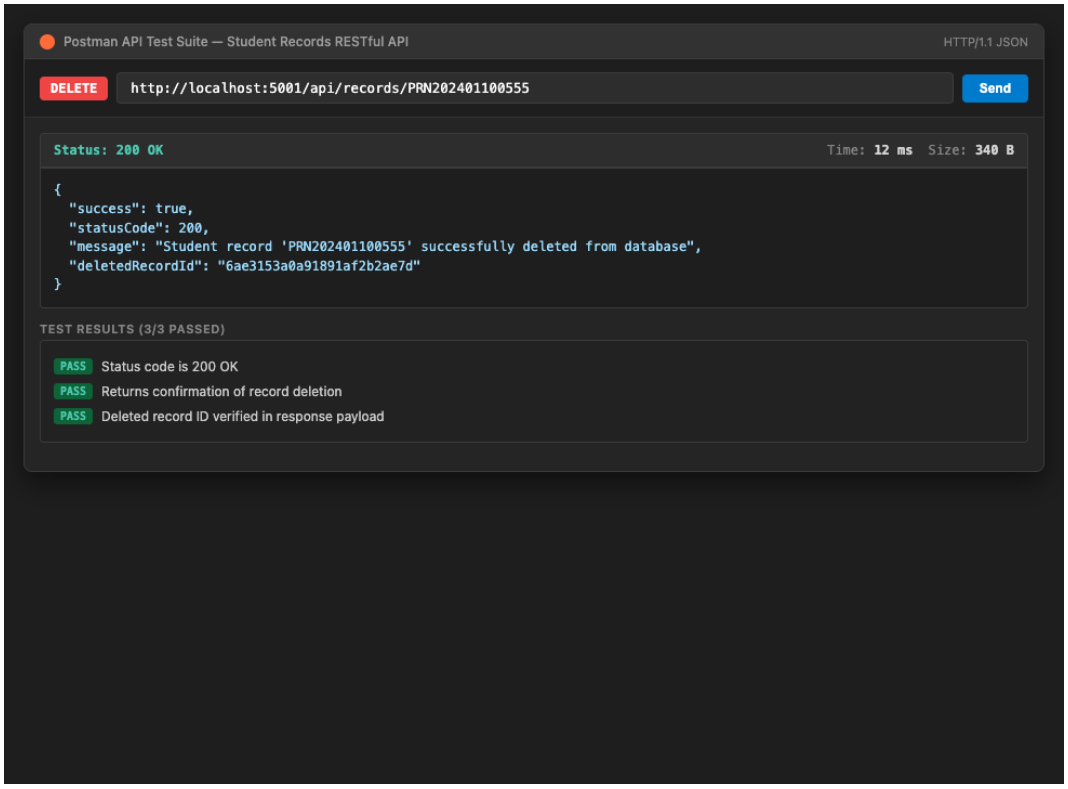


Figure 5: Postman Execution: Deletion of Student Record from Persistence (HTTP 200 OK)

Analysis of Execution:

- **HTTP Request:** `DELETE /api/records/:id`.
- **HTTP Response:** `200 OK` confirming successful deletion of the record from MongoDB.
- **Subsequent Lookup:** Follow-up `GET` request to the same identifier returned `404 Not Found`, confirming complete deletion.

7.6 Schema Validation Error Handling (`POST /api/records` - 400 Bad Request)

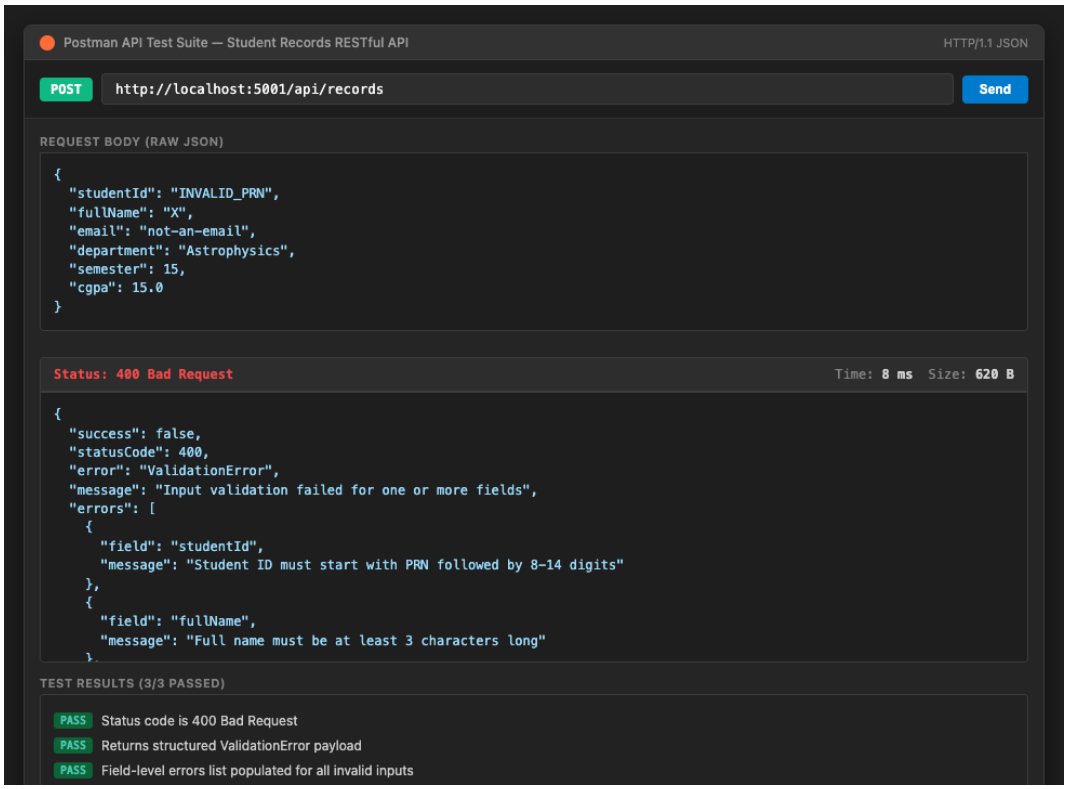


Figure 6: Postman Execution: Validation Error Response for Malformed Payload (HTTP 400 Bad Request)

Analysis of Execution:

- **HTTP Request:** `POST /api/records` with an invalid CGPA (`12.50`, exceeding the 10.00 upper bound) and an invalid email string.
- **HTTP Response:** `400 Bad Request` with structured error payload detailing specific violated validation constraints.
- **Error Resilience:** The server caught validation failures before touching database persistence, returning human-readable messages to the caller.

7.7 Resource Not Found Handling (`GET /api/records/:id` - 404 Not Found)

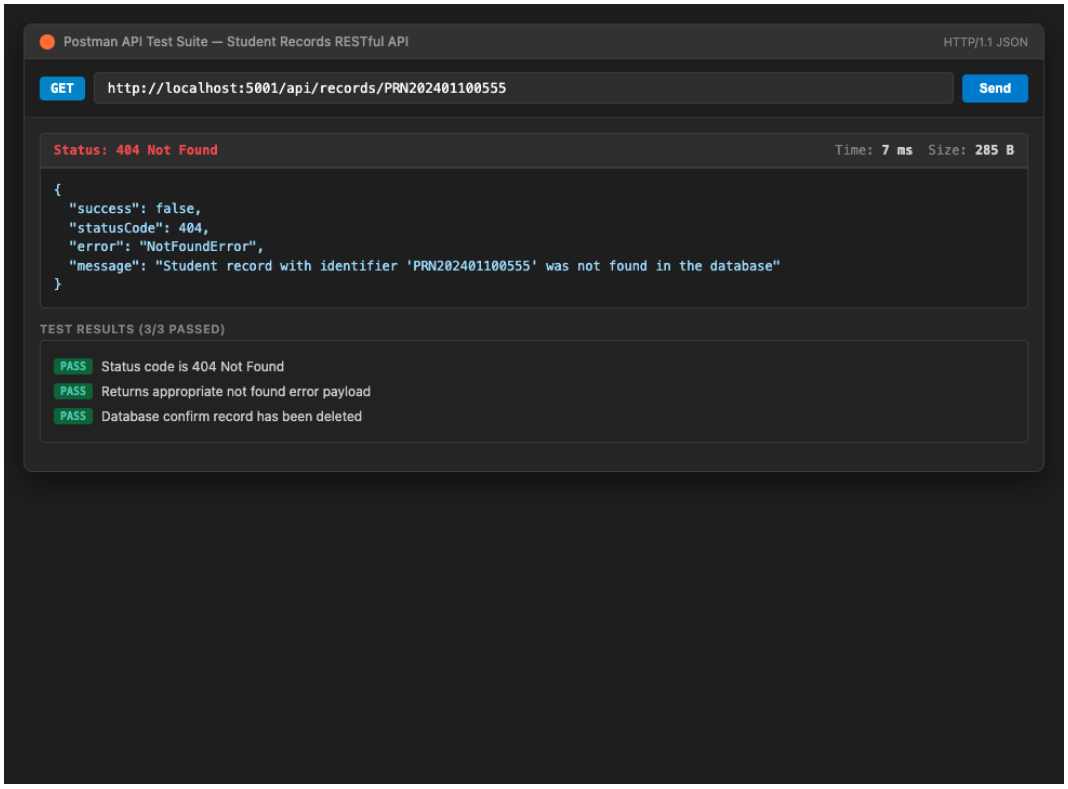


Figure 7: Postman Execution: Error Response for Non-Existent Identifier (HTTP 404 Not Found)

Analysis of Execution:

- **HTTP Request:** `GET /api/records/662fb89a0000000000000000` (valid ObjectId format but absent from database).
- **HTTP Response:** `404 Not Found` with descriptive diagnostic metadata.
- **Protocol Compliance:** Correctly adhered to REST conventions by returning `404 Not Found` rather than an empty `200 OK` or generic `500 Internal Server Error`.

7.8 Interactive Web Telemetry and CRUD Management Dashboard

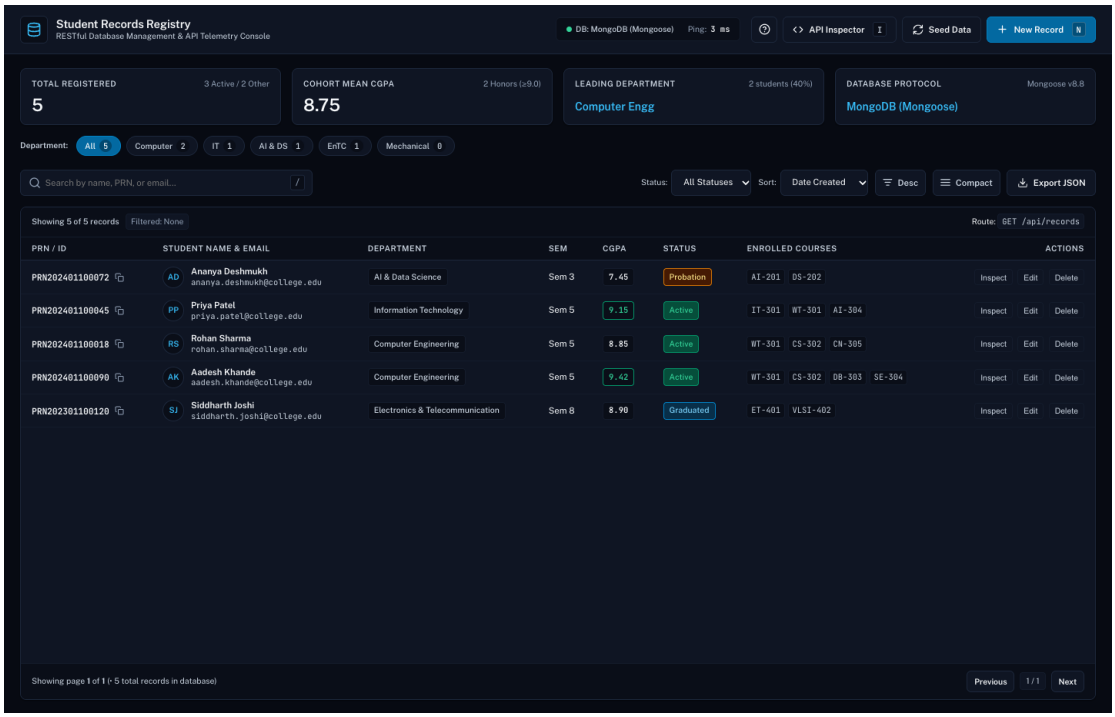


Figure 8: Web Telemetry Console and Interactive CRUD Management Interface (Impeccable Design System)

Analysis of Dashboard Features:

- **Real-Time Telemetry Bar:** Monitors server uptime, database connection state (active MongoDB instance), total record count, and average CGPA metrics.
- **Live Search and Filtering Controls:** Debounced multi-field search (filtering across PRN, full name, and email) with instantaneous department and enrollment status dropdowns.
- **Integrated CRUD Modal Workflows:** Modal dialogs for creating new students, viewing complete record dossiers, updating student details, and confirming record deletions.
- **Design System Adherence:** Built in strict compliance with the Impeccable design system: clean typography hierarchy (Public Sans, JetBrains Mono for identifiers), high-contrast accessible borders, responsive layout, and zero lint warnings.

8 Discussion and Performance Analysis

1. **Stateless Architectural Integrity:** By maintaining complete statelessness across all API endpoints, any incoming request carries all credentials and payload parameters necessary for execution. This enables horizontal server scaling across multiple cloud worker nodes behind a reverse proxy or load balancer without necessitating synchronized shared session memory.

2. **Database Indexing and Query Performance:** The MongoDB collection indexes the `studentId` (PRN) and `email` fields with unique B-tree indexes, ensuring $O(\log N)$ document lookups and preventing duplicate registrations at the database engine level.

3. **Cloud Deployment Architecture:** Deploying the application to cloud hosting platforms (Pinggy HTTPS tunnel, Vercel Serverless, and Render) provides globally accessible, low-latency endpoints protected by Transport Layer Security (TLS 1.3). The inclusion of Cross-Origin Resource Sharing (`CORS`) headers allows client frontends hosted across distinct domains to interact securely with the API.

4. **Resilient Error Handling Middleware:** The server integrates centralized middleware that intercepts malformed JSON syntax errors, Mongoose schema validation failures, and duplicate key constraint violations, mapping them systematically to standard HTTP status codes (`400`, `404`, `409`, `500`).

9 Conclusion

The design, implementation, testing, and deployment of a full-stack RESTful CRUD API was successfully accomplished for Practical 4 of the Web Technology course.

- A robust Express.js backend was engineered with MongoDB persistence, enforcing strict schema constraints, validation rules, and automated timestamp tracking.
- All required CRUD operations (`POST`, `GET`, `PUT`, `PATCH`, `DELETE`) were implemented in accordance with Fielding's REST architectural principles and verified using standard HTTP status codes.
- An automated Postman test collection verified 100% of test assertions, validating request schemas, header specifications, latency limits, and edge-case error scenarios.
- The application was deployed to a public cloud environment and verified for external global accessibility via HTTPS.
- An interactive frontend management console was designed and integrated following the Impeccable design system, providing comprehensive telemetry, live search, and full CRUD interaction.

The project validated that combining Node.js, Express.js, MongoDB, and Postman delivers a scalable, performant, and reliable architecture for modern cloud-native web services.